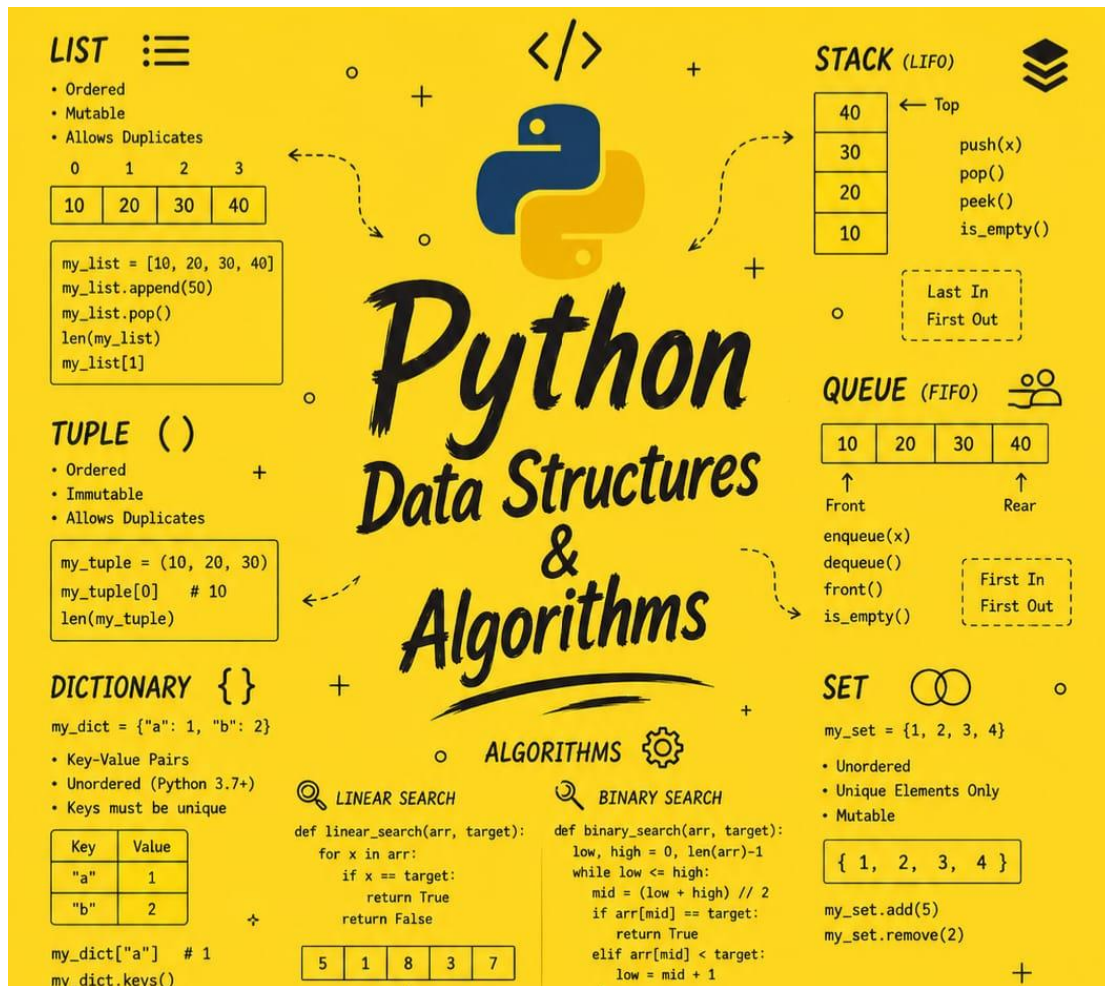




Python Data Structures & Algorithms

By Jawad Jammoul

Content:

Chapter 13 : LinkedList	Page
	1
Chapter 14 : Stack & Queue	9
Chapter 15 : Algorithm Analysis	18
Chapter 16 : Recursion	22
Chapter 17 : Trees	23
Chapter 18 : Graphs	30
Chapter 19 : HashTable & Dictionary	35

Chapter 13 : LinkedList

01*Definition:

A Linked List is a linear data structure made of connected nodes. Each node stores data and a reference to the next node. Unlike arrays, linked lists do not store elements in contiguous memory.

02*Why Using LinkedList?

Dynamic size (can grow or shrink easily).

Fast insertion and deletion.

No need for contiguous memory.

Useful when the number of elements changes frequently.

03*Advantages:

Dynamic memory allocation.

Efficient insertion and deletion.

Memory is allocated only when needed.

04*Disadvantages:

Slower access to elements (must traverse from the beginning).

Uses extra memory for pointers.

More complex than arrays.

Each Node Consists of Two Parts

Every node in a singly linked list contains:

Data: Stores the value.

Next: Stores the reference (pointer) to the next node. The last node points to None.

05*NodeData Class:

The NodeData class stores the value of a node. It represents the data contained in a linked list node.

```
class NodeData:
    def __init__(self, data):
        self.data = data
```

06*Node Class:

The Node class represents a single node in a linked list. It contains the node data and a reference to the next node.

```
class Node:
    def __init__(self, data):
        self.data = NodeData(data)
        self.next = None
```

07*LinkedList Class:

The LinkedList class manages all nodes in the linked list. It starts with a head pointer, which refers to the first node. If the list is empty, head is None.

```
class LinkedList:
    def __init__(self):
        self.head = None
```

08*LinkedList Methods:

display()

Displays all elements of the linked list from the head to the last node.

```
def display(self):
    if self.head is None:
        print("Linked List is empty.")
        return
    current = self.head
    while current:
        print(current.data.data, end=" -> ")
        current = current.next
    print("None")
```

is_empty()

Checks whether the linked list is empty.

```
def is_empty(self):
    return self.head is None
```

count_nodes()

Counts and returns the total number of nodes in the linked list.

```
def count_nodes(self):
    count = 0
    current = self.head
    while current:
        count += 1
        current = current.next
    return count
```

contains()

Searches for a given value in the linked list. Returns True if the value exists; otherwise, returns False.

```
def contains(self, value):
    current = self.head
    while current:
        if current.data.data == value:
            return True
        current = current.next
    return False
```

add_head()

Inserts a new node at the beginning of the linked list.

```
def add_head(self, value):
    new_node = Node(value)
    new_node.next = self.head
    self.head = new_node
```

add_tail()

Inserts a new node at the end of the linked list.

```
def add_tail(self, value):
    new_node = Node(value)
    if self.head is None:
        self.head = new_node
        return
    current = self.head
    while current.next:
        current = current.next
    current.next = new_node
```

insert_at_position()

Inserts a new node at a specified position.

```
def insert_at_position(self, value, position):
    if position == 0:
        self.add_head(value)
        return
    new_node = Node(value)
    current = self.head
    for _ in range(position - 1):
        if current is None:
            return
        current = current.next
    if current is None:
        return
    new_node.next = current.next
    current.next = new_node
```

equals()

Compares two linked lists and returns True if they contain the same elements in the same order.

```
def equals(self, other):
    first = self.head
    second = other.head
    while first and second:
        if first.data.data != second.data.data:
            return False
        first = first.next
        second = second.next
    return first is None and second is None
```

insert_sorted()

Inserts a new node into its correct position while keeping the list sorted.

```
def insert_sorted(self, value):
    new_node = Node(value)
    if self.head is None or self.head.data.data >= value:
        new_node.next = self.head
        self.head = new_node
        return
    current = self.head
    while current.next and current.next.data.data < value:
        current = current.next
    new_node.next = current.next
    current.next = new_node
```

copy()

Creates and returns a copy of the linked list.

```
def copy(self):
    new_list = LinkedList()
    current = self.head
    while current:
        new_list.add_tail(current.data.data)
        current = current.next
    return new_list
```

reverse()

Reverses the order of all nodes in the linked list.

```
def reverse(self):
    previous = None
    current = self.head
    while current:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node
    self.head = previous
```

is_palindrome()

Checks whether the linked list is a palindrome.

```
def is_palindrome(self):
    values = []
    current = self.head
    while current:
        values.append(current.data.data)
        current = current.next
    return values == values[::-1]
```

merge()

Merges another linked list with the current linked list.

```
def merge(self, other):
    if self.head is None:
        self.head = other.head
        return
    current = self.head
    while current.next:
        current = current.next
    current.next = other.head
```

delete_at_position()

Deletes the node at a specified position.

```
def delete_at_position(self, position):
    if self.head is None:
        return
    if position == 0:
        self.head = self.head.next
        return
    current = self.head
    for _ in range(position - 1):
        if current.next is None:
            return
        current = current.next
    if current.next:
        current.next = current.next.next
```

delete_head()

Removes the first node from the linked list.

```
def delete_head(self):
    if self.head:
        self.head = self.head.next
```

delete_tail()

Removes the last node from the linked list.

```
def delete_tail(self):
    if self.head is None:
        return
    if self.head.next is None:
        self.head = None
        return
    current = self.head
    while current.next.next:
        current = current.next
    current.next = None
```

09*Application Class (LinkedList):

```
class Application:
    def main():
        linked_list = LinkedList()
        linked_list.add_head(10)
        linked_list.add_tail(20)
        linked_list.add_tail(30)
        linked_list.display()
        print("Number of Nodes:", linked_list.count_nodes())
        print("Contains 20:", linked_list.contains(20))
Application.main()
```

10*Doubly Linked List:

A Doubly Linked List (DLL) is a linear data structure where each node contains data and two references: one to the next node and one to the previous node. This allows traversal in both forward and backward directions.

11*Why Using Doubly Linked List?

Allows traversal in both directions.

Faster insertion and deletion when the node is known.

Simplifies operations such as reverse traversal.

Useful for navigation systems, browser history, and undo/redo features.

12*Advantages:

Supports forward and backward traversal.
Efficient insertion and deletion.
Easier to reverse the list.
More flexible than a singly linked list.

13*Disadvantages:

Requires extra memory for the previous pointer.
More complex to implement.
Insertion and deletion require updating two links instead of one.

14*Doubly Linked List Structures:

NodeData Class

```
class NodeData:  
    def __init__(self, data):  
        self.data = data
```

Node Class

```
class Node:  
    def __init__(self, data):  
        self.data = NodeData(data)  
        self.next = None  
        self.prev = None
```

DoublyLinkedList Class

```
class DoublyLinkedList:  
    def __init__(self):  
        self.head = None  
        self.tail = None
```

15*Some Methods in DoublyLinkedList Class:

display_forward()

Displays all nodes from the head to the tail.

```
def display_forward(self):
    current = self.head
    while current:
        print(current.data.data, end=" <-> ")
        current = current.next
    print("None")
```

display_backward()

Displays all nodes from the tail to the head.

```
def display_backward(self):
    current = self.tail
    while current:
        print(current.data.data, end=" <-> ")
        current = current.prev
    print("None")
```

add_head()

Inserts a new node at the beginning of the doubly linked list.

```
def add_head(self, value):
    new_node = Node(value)
    if self.head is None:
        self.head = self.tail = new_node
        return
    new_node.next = self.head
    self.head.prev = new_node
    self.head = new_node
```

add_tail()

Inserts a new node at the end of the doubly linked list.

```
def add_tail(self, value):
    new_node = Node(value)
    if self.tail is None:
        self.head = self.tail = new_node
        return
    self.tail.next = new_node
    new_node.prev = self.tail
    self.tail = new_node
```

delete_head()

Removes the first node from the doubly linked list.

```
def delete_head(self):
    if self.head is None:
        return
    if self.head == self.tail:
        self.head = self.tail = None
        return
    self.head = self.head.next
    self.head.prev = None
```

delete_tail()

Removes the last node from the doubly linked list.

```
def delete_tail(self):
    if self.tail is None:
        return
    if self.head == self.tail:
        self.head = self.tail = None
        return
    self.tail = self.tail.prev
    self.tail.next = None
```

is_empty()

Checks whether the doubly linked list is empty.

```
def is_empty(self):
    return self.head is None
```

Chapter 14 : Stack & Queue

01*ADT (Abstract Data Type):

An Abstract Data Type (ADT) is a logical model that defines what a data structure can do, without specifying how it is implemented.

Characteristics:

- Defines the data and allowed operations.
- Hides implementation details.
- Can be implemented using different data structures.

Examples:

Stack: push(), pop(), peek(), is_empty()

Queue: enqueue(), dequeue(), front(), is_empty()

Advantages:

- Simplifies program design.
- Promotes code reusability.
- Improves maintainability.
- Allows multiple implementations for the same ADT.

02*Stack – Array Implementation:

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle. The last element added is the first one to be removed.

Initializes the stack with a fixed size and sets the top index to -1.

```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1
```

Attributes:

size: Maximum number of elements.

stack: Array used to store elements.

top: Index of the top element (-1 when the stack is empty).

push(): Adds a new element to the top of the stack.

```
def push(self, value):
    if self.is_full():
        print("Stack Overflow")
        return
    self.top += 1
    self.stack[self.top] = value
```

pop()

Removes and returns the top element of the stack.

```
def pop(self):
    if self.is_empty():
        print("Stack Underflow")
        return None
    value = self.stack[self.top]
    self.top -= 1
    return value
```

top()

Returns the top element without removing it.

```
def top_element(self):
    if self.is_empty():
        return None
    return self.stack[self.top]
```

is_empty()

Checks whether the stack is empty.

```
def is_empty(self):
    return self.top == -1
```

is_full()

Checks whether the stack is full.

```
def is_full(self):
    return self.top == self.size - 1
```

Example Usage:

```
stack = Stack(5)
stack.push(10)
stack.push(20)
stack.push(30)
print("Top:", stack.top_element())
print("Popped:", stack.pop())
print("Top after pop:", stack.top_element())
print("Is Empty?", stack.is_empty())
print("Is Full?", stack.is_full())
```

Output:

```
Top: 30
Popped: 30
Top after pop: 20
Is Empty? False
Is Full? False
```

03*Stack-LinkedList Implementation:

A Stack can be implemented using a Linked List. The head node represents the top of the stack, allowing efficient insertion and deletion in $O(1)$ time.

Attributes:

head: Points to the top node of the stack.

Initializes an empty stack.

```
class Stack:
    def __init__(self):
        self.head = None
```

NodeData Class:

The NodeData class stores the value of a node.

```
class NodeData:
    def __init__(self, data):
        self.data = data
```

Node Class:

The Node class represents a node in the linked-list stack. Each node contains data and a reference to the next node.

```
class Node:
    def __init__(self, data):
        self.data = NodeData(data)
        self.next = None
```

Stack Methods

push()

Adds a new element to the top of the stack.

```
def push(self, value):
    new_node = Node(value)
    new_node.next = self.head
    self.head = new_node
```

pop()

Removes and returns the top element of the stack.

```
def pop(self):
    if self.is_empty():
        return None
    value = self.head.data.data
    self.head = self.head.next
    return value
```

peek()

Returns the top element without removing it.

```
def peek(self):
    if self.is_empty():
        return None
    return self.head.data.data
```

is_empty()

Checks whether the stack is empty.

```
def is_empty(self):
    return self.head is None
display()
```

Example Usage:

```
stack = Stack()
stack.push(10)
stack.push(20)
stack.push(30)
stack.display()
print("Top:", stack.peek())
print("Popped:", stack.pop())
print("Top after pop:", stack.peek())
```

Output:

```
30
20
10
Top: 30
Popped: 30
Top after pop: 20
```

04*Advantages of Using Stack: (Linked List)

- Dynamic size (no fixed capacity).
- Efficient push and pop operations in $O(1)$ time.
- No stack overflow unless memory is exhausted.
- Memory is allocated only when needed.
- Easy to implement and maintain.
- Suitable for applications with frequent insertions and deletions.

05*Implementing a Queue Using Array (Circular Buffer)

A Queue is a linear data structure that follows the FIFO (First In, First Out) principle. A Circular Buffer reuses empty spaces in the array by wrapping around from the last index to the first.

Initializes an empty queue with a fixed size.

```
class Queue:
    def __init__(self, size):
        self.queueArray = [None] * size
        self.head = 0
        self.tail = -1
        self._size = 0
        self.capacity = size
```

Attributes:

queueArray: Array used to store queue elements.

head: Index of the first element.

tail: Index of the last element.

_size: Current number of elements.

capacity: Maximum queue size.

isEmpty()

Checks whether the queue is empty.

```
def isEmpty(self):
    return self._size == 0
```

isFull()

Checks whether the queue is full.

```
def isFull(self):
    return self._size == self.capacity
```

enqueue()

Adds a new element to the rear of the queue.

```
def enqueue(self, value):
    if self.isFull():
        print("Queue Overflow")
        return
    self.tail = (self.tail + 1) % self.capacity
    self.queueArray[self.tail] = value
    self._size += 1
```

dequeue()

Removes and returns the front element of the queue.

```
def dequeue(self):
    if self.isEmpty():
        print("Queue Underflow")
        return None
    value = self.queueArray[self.head]
    self.queueArray[self.head] = None
    self.head = (self.head + 1) % self.capacity
    self._size -= 1
    return value
```

headValue()

Returns the first element without removing it.

```
def headValue(self):
    if self.isEmpty():
        return None
    return self.queueArray[self.head]
```

tailValue()

Returns the last element without removing it.

```
def tailValue(self):
    if self.isEmpty():
        return None
    return self.queueArray[self.tail]
```

print()

Displays all queue elements from head to tail.

```
def print(self):
    if self.isEmpty():
        print("Queue is empty.")
        return
    index = self.head
    for _ in range(self._size):
        print(self.queueArray[index], end=" ")
        index = (index + 1) % self.capacity
```

Example Usage:

```
queue = Queue(5)
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
queue.print()
print("Head:", queue.headValue())
print("Tail:", queue.tailValue())
print("Dequeued:", queue.dequeue())
queue.print()
```

Output:

```
10 20 30
Head: 10
Tail: 30
Dequeued: 10
20 30
```

06*Queue LinkedList Implementation:

A Queue can be implemented using a Linked List. It uses two pointers: head (front) and tail (rear), allowing efficient enqueue and dequeue operations in $O(1)$ time.

NodeData Class:

The NodeData class stores the value of a node.

```
class NodeData:
    def __init__(self, data):
        self.data = data
```

Node Class:

The Node class represents a node in the queue. Each node contains data and a reference to the next node.

```
class Node:
    def __init__(self, data):
        self.data = NodeData(data)
        self.next = None
```

Queue Class:

The Queue class manages the linked-list queue using two pointers: head and tail.

```
class Queue:
    def __init__(self):
        self.head = None
        self.tail = None
```

Attributes:

head: Points to the first node (front).

tail: Points to the last node (rear).

isEmpty()

Checks whether the queue is empty.

```
def isEmpty(self):  
    return self.head is None
```

enqueue()

Adds a new element to the rear of the queue.

```
def enqueue(self, value):  
    new_node = Node(value)  
    if self.isEmpty():  
        self.head = self.tail = new_node  
    return  
    self.tail.next = new_node  
    self.tail = new_node
```

dequeue()

Removes and returns the front element.

```
def dequeue(self):  
    if self.isEmpty():  
        return None  
    value = self.head.data.data  
    self.head = self.head.next  
    if self.head is None:  
        self.tail = None  
    return value
```

headValue()

Returns the front element without removing it.

```
def headValue(self):
    if self.isEmpty():
        return None
    return self.head.data.data
```

tailValue()

Returns the rear element without removing it.

```
def tailValue(self):
    if self.isEmpty():
        return None
    return self.tail.data.data
```

print()

Displays all queue elements from front to rear.

```
def print(self):
    current = self.head
    while current:
        print(current.data.data, end=" ")
        current = current.next
    print()
```

Example Usage:

```
queue = Queue()
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
queue.print()
print("Head:", queue.headValue())
print("Tail:", queue.tailValue())
print("Dequeued:", queue.dequeue())
queue.print()
```

Output:

```
10 20 30
Head: 10
Tail: 30
Dequeued: 10
20 30
```

Chapter 15 : Algorithm Analysis

01*What is an Algorithm?

An Algorithm is a finite sequence of well-defined steps used to solve a problem or perform a specific task.

Characteristics:

- Clear and unambiguous.
- Finite number of steps.
- Produces a correct output.
- Efficient and easy to understand.

02*Algorithm Analysis:

Algorithm Analysis is the process of evaluating an algorithm's performance by measuring the time and memory it requires.

Why Analyze Algorithms?

- Compare different algorithms.
- Choose the most efficient solution.
- Improve program performance.
- Reduce execution time and memory usage.

03*Running Time:

Running Time is the amount of time an algorithm takes to complete its execution. It usually depends on the size of the input, denoted by n .

Example:

- Searching one element in a list.
- Sorting a list of numbers.
- Finding the largest value in an array.

04*Experimental Study:

Experimental Study measures an algorithm's actual execution time by running it on a computer with different input sizes.

Example:

```
import time
numbers = list(range(100000))
start = time.time()
total = sum(numbers)
end = time.time()
print("Execution Time:", end - start,
      "seconds")
```

Sample Output:

```
Execution Time: 0.0012 seconds
```

05*Measuring Running Time Experimentally:

Running time can be measured by recording the execution time before and after running an algorithm.

```
import time
def find_max(numbers):
    maximum = numbers[0]
    for num in numbers:
        if num > maximum:
            maximum = num
    return maximum
data = list(range(100000))
start = time.time()
find_max(data)
end = time.time()
print("Running Time:", end - start, "seconds")
```

06*Challenges of Experimental Analysis:

Experimental analysis measures actual execution time, but the results may vary under different conditions.

Challenges:

Depends on computer hardware.

Affected by the operating system.

Programming language influences performance.

Different input data may produce different results.

Execution time may vary between runs.

07*Measuring Operations as a Function of Input Size:

Instead of measuring execution time, algorithm analysis counts the number of operations performed as the input size (n) increases.

Advantages:

- Independent of hardware.
- Independent of programming language.
- Makes algorithm comparison easier.
- Forms the basis of complexity analysis.

08*Best, Worst, and Average Case:

Best Case:

The minimum number of operations an algorithm performs.

Example: Searching for the first element in a list.

Worst Case:

The maximum number of operations an algorithm performs.

Example: Searching for the last element in a list.

Average Case:

The expected number of operations for a typical input.

Example: Searching for an element located near the middle of a list.

09*The Seven Functions Used in Algorithm Analysis:

These functions describe how an algorithm's running time grows as the input size (n) increases.

$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Linearithmic
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential

10*Comparing Growth Rates:

Algorithms with lower growth rates are generally more efficient for large input sizes.

From fastest to slowest:

$O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2) \rightarrow O(n^3) \rightarrow O(2^n)$

11*Asymptotic Analysis:

Asymptotic Analysis evaluates an algorithm by studying how its running time or memory usage grows as the input size (n) becomes very large.

Advantages:

- Independent of hardware.
- Independent of programming language.
- Compares algorithms fairly.
- Predicts performance for large inputs.

12*Big O Notation:

Big O Notation describes the worst-case running time of an algorithm. It shows how the running time grows as the input size increases.

Examples:

Accessing an array element : $O(1)$

Binary Search : $O(\log n)$

Linear Search : $O(n)$

Merge Sort : $O(n \log n)$

Bubble Sort : $O(n^2)$

Triple Nested Loops : $O(n^3)$

Recursive Fibonacci : $O(2^n)$

13*Comparative Analysis:

Comparative Analysis is the process of comparing two or more algorithms to determine which one is more efficient.

- Comparison Criteria
- Running time.
- Memory usage.
- Time complexity.
- Space complexity.
- Scalability for large input sizes.

14*Examples of Algorithm Analysis:

Example 1: Constant Time – $O(1)$

```
numbers = [10, 20, 30, 40, 50]
print(numbers[2])
```

Example 2: Linear Time – $O(n)$

```
numbers = [10, 20, 30, 40, 50]
for num in numbers:
    print(num)
```

Example 3: Quadratic Time – $O(n^2)$

```
numbers = [1, 2, 3]
for i in numbers:
    for j in numbers:
        print(i, j)
```

Chapter 16 : Recursion

01*What is Recursion?

Recursion is a programming technique in which a function calls itself to solve a problem by breaking it into smaller subproblems.

Characteristics:

- A function calls itself.
- Solves a problem step by step.
- Stops when a base case is reached.
- Simplifies problems with repetitive patterns.

02*Components of a Recursive Method:

A recursive method consists of two essential parts:

A)Base Case:

The Base Case is the stopping condition that ends the recursion. Without it, the function would continue calling itself indefinitely.

Example:

```
def countdown(n):  
    if n == 0:  
        return  
    print(n)  
    countdown(n - 1)
```

Here, `n == 0` is the Base Case.

B)Recursive Case:

The Recursive Case is the part where the function calls itself with a smaller or simpler input until the base case is reached.

Example:

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

Here, `factorial(n - 1)` is the Recursive Case.

03*Example : Factorial Using Recursion

The factorial of a positive integer `n` is the product of all positive integers from 1 to `n`.

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial(n - 1)  
print(factorial(5))
```

Output:

120

04*Void vs Non-Void Recursive Methods:

Void Recursive Method:

A void recursive method performs a task but does not return a value.

```
def countdown(n):  
    if n == 0:  
        return  
    print(n)  
    countdown(n - 1)  
countdown(5)
```

Output:

```
5  
4  
3  
2  
1
```

Non-Void Recursive Method:

A non-void recursive method returns a value to the caller after the recursive calls are completed.

```
def sum_numbers(n):  
    if n == 0:  
        return 0  
    return n + sum_numbers(n - 1)  
print(sum_numbers(5))
```

Output:

```
15
```

Chapter 17 : Trees

01*Tree Definitions:

- 1- **Root** : The root is the topmost node of a tree. It has no parent.
- 2- **Parent** : A parent is a node that has one or more child nodes.
- 3- **Child** : A child is a node directly connected below its parent.
- 4- **Leaf** : A leaf is a node that has no children.
- 5- **Sibling** : Siblings are nodes that share the same parent.
- 6- **Branch** : A branch is a path connecting nodes in a tree.
- 7- **Level** : The level of a node is its distance from the root. The root is at Level 0.
- 8- **Height** : The height of a tree is the number of edges on the longest path from the root to a leaf.
- 9- **Degree** : The degree of a node is the number of its children. The degree of a tree is the maximum degree of any node.
- 10- **Weight** : The weight is a value assigned to a node or an edge, depending on the application.
- 11- **Moment** : The moment (depth) of a node is the number of edges from the root to that node.

02*Binary Tree:

A Binary Tree is a tree in which each node has at most two children, called the left child and the right child.

Traversing a Binary Tree

(a)Pre-order Traversal

Order: Root → Left → Right

```
def preorder(root):  
    if root is None:  
        return  
    print(root.data, end=" ")  
    preorder(root.left)  
    preorder(root.right)
```

(b)In-order Traversal

Order: Left → Root → Right

```
def inorder(root):  
    if root is None:  
        return  
    inorder(root.left)  
    print(root.data, end=" ")  
    inorder(root.right)
```

(c)Post-order Traversal

Order: Left → Right → Root

Visits the Left subtree, then the Right subtree, then the Root.

```
def postorder(root):  
    if root is None:  
        return  
    postorder(root.left)  
    postorder(root.right)  
    print(root.data, end=" ")
```

How to Represent a Binary Tree

Node Class

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

Creating a Binary Tree

```
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(3)
root.left.right = Node(7)
root.right.left = Node(12)
root.right.right = Node(20)
```

Tree Structure

```
      10
     /  \
    5    15
   /\  /\
  3 7 12 20
```

03*Binary Search Tree (BST):

A Binary Search Tree (BST) is a binary tree in which:

All values in the left subtree are less than the root.

All values in the right subtree are greater than the root.

Both left and right subtrees are also Binary Search Trees.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BST:
    def __init__(self):
        self.root = None
    def insert(self, root, value):
        if root is None:
            return Node(value)
        if value < root.data:
            root.left = self.insert(root.left, value)
        else:
            root.right = self.insert(root.right, value)
        return root
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)
```

Example:

```
tree = BST()
values = [50, 30, 70, 20, 40, 60, 80]
for value in values:
    tree.root = tree.insert(tree.root, value)
tree.inorder(tree.root)
```

Output:

```
20 30 40 50 60 70 80
```

04*Level Order Traversal: (Breadth-First Traversal)

Level Order Traversal visits the nodes level by level, starting from the root and moving from left to right. It uses a Queue to keep track of the nodes.

```
from collections import deque
def level_order(root):
    if root is None:
        return
    queue = deque([root])
    while queue:
        current = queue.popleft()
        print(current.data, end=" ")
        if current.left:
            queue.append(current.left)
        if current.right:
            queue.append(current.right)
```

Example

```
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(3)
root.left.right = Node(7)
root.right.left = Node(12)
root.right.right = Node(20)
level_order(root)
```

Output

```
10 5 15 3 7 12
20
```

05*Some Useful Tree Functions:

Count Total Nodes

Returns the total number of nodes in the binary tree.

```
def count_nodes(root):
    if root is None:
        return 0
    return 1 + count_nodes(root.left) +
count_nodes(root.right)
```

Find Height of Tree

Returns the height of the binary tree.

```
def tree_height(root):
    if root is None:
        return -1
    left_height = tree_height(root.left)
    right_height = tree_height(root.right)
    return 1 + max(left_height,
right_height)
```

Check if Tree is Balanced

Checks whether the binary tree is height-balanced. A tree is balanced if the height difference between the left and right subtrees of every node is at most 1.

```
def is_balanced(root):
    if root is None:
        return True
    left_height = tree_height(root.left)
    right_height = tree_height(root.right)
    if abs(left_height - right_height) > 1:
        return False
    return is_balanced(root.left) and
is_balanced(root.right)
```

Example

```
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(3)
root.left.right = Node(7)
print("Total Nodes:", count_nodes(root))
print("Height:", tree_height(root))
print("Balanced:", is_balanced(root))
```

Output

```
Total Nodes: 5
Height: 2
Balanced: True
```

06*BST Deletion:

BST Deletion removes a node from a Binary Search Tree while preserving the BST property.

Case 1: The node is a leaf (no children). Remove it directly.

Case 2: The node has one child. Replace it with its child.

Case 3: The node has two children. Replace it with its in-order successor (smallest value in the right subtree), then delete the successor.

```

class BST:
    ...
    def min_value(self, node):
        current = node
        while current.left:
            current = current.left
        return current
    def delete(self, root, value):
        if root is None:
            return root
        if value < root.data:
            root.left = self.delete(root.left, value)
        elif value > root.data:
            root.right = self.delete(root.right, value)
        else:
            if root.left is None:
                return root.right
            if root.right is None:
                return root.left
            temp = self.min_value(root.right)
            root.data = temp.data
            root.right = self.delete(root.right, temp.data)
        return root

```

Example

```

tree = BST()
values = [50, 30, 70, 20, 40, 60, 80]
for value in values:
    tree.root = tree.insert(tree.root, value)
tree.root = tree.delete(tree.root, 30)
tree.inorder(tree.root)

```

Output

```
20 40 50 60 70 80
```

Chapter 18 : Graphs

01*What is a Graph?

A Graph is a non-linear data structure consisting of a set of vertices (nodes) and edges that connect the vertices.

A graph is represented as:

$G = (V, E)$

where:

V = Set of vertices.

E = Set of edges.

02*Types of Graph:

Undirected Graph

An edge connects two vertices in both directions.

Example: A — B

Directed Graph (Digraph)

Each edge has a direction.

Example: A → B

Weighted Graph

Each edge has an associated weight (cost or distance).

Unweighted Graph

Edges have no weights.

Connected Graph

There is a path between every pair of vertices.

Disconnected Graph

At least one vertex cannot be reached from another vertex.

03*Graph Representation:

Graphs are commonly represented using an Adjacency Matrix or an Adjacency List.

A)Adjacency Matrix:

An Adjacency Matrix is a two-dimensional array where rows and columns represent vertices.

A value of 1 indicates an edge exists, while 0 indicates no edge.

```
graph = [  
    [0, 1, 1, 0],  
    [1, 0, 1, 0],  
    [1, 1, 0, 1],  
    [0, 0, 1, 0]  
]  
for row in graph:  
    print(row)
```

Output:

```
[0, 1, 1, 0]  
[1, 0, 1, 0]  
[1, 1, 0, 1]  
[0, 0, 1, 0]
```

B)Adjacency List:

An Adjacency List stores each vertex along with a list of its adjacent vertices.

```
graph = {  
    "A": ["B", "C"],  
    "B": ["A", "C"],  
    "C": ["A", "B", "D"],  
    "D": ["C"]  
}  
for vertex in graph:  
    print(vertex, "->", graph[vertex])
```

Output:

```
A -> ['B', 'C']  
B -> ['A', 'C']  
C -> ['A', 'B', 'D']  
D -> ['C']
```

04*Graph Terminologies:

Vertex:

A vertex (node) is a point in a graph.

Edge:

An edge is a connection between two vertices.

Path:

A path is a sequence of vertices connected by edges.

Degree:

The degree of a vertex is the number of edges connected to it.

Adjacent Vertices:

Two vertices are adjacent if they are directly connected by an edge.

05*Graph Traversal:

Graph traversal is the process of visiting every vertex in a graph.

A)Depth First Search (DFS):

Depth First Search (DFS) visits a vertex, then explores one branch as deeply as possible before backtracking.

```
def dfs(graph, vertex, visited):
    if vertex not in visited:
        print(vertex, end=" ")
        visited.add(vertex)
        for neighbor in graph[vertex]:
            dfs(graph, neighbor, visited)
```

Example:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
visited = set()
dfs(graph, 'A', visited)
```

Output:

```
A B D E C F
```

B) Breadth First Search (BFS):

Breadth First Search (BFS) visits all neighboring vertices first, then moves to the next level. It uses a Queue.

```
from collections import deque
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Example:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
bfs(graph, 'A')
```

Output:

A B C D E F

06*Weighted Graph Example:

```
graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('A', 4), ('C', 5), ('D', 10)],
    'C': [('A', 2), ('B', 5), ('D', 3)],
    'D': [('B', 10), ('C', 3)]
}
for vertex in graph:
    print(vertex, "->", graph[vertex])
```

Output:

```
A -> [('B', 4), ('C', 2)]
B -> [('A', 4), ('C', 5), ('D', 10)]
C -> [('A', 2), ('B', 5), ('D', 3)]
D -> [('B', 10), ('C', 3)]
```

07*Applications of Graph:

Graphs are widely used in many real-world applications.

- Social networks.
- GPS and navigation systems.
- Computer networks.
- Web page linking.
- Airline and railway routes.
- Recommendation systems.
- Network routing algorithms.
- Project scheduling.

08*Graph Algorithms:

Definition:

Graph Algorithms are algorithms used to solve problems on graphs, such as searching, finding paths, and building minimum-cost networks.

Common Graph Algorithms:

Depth First Search (DFS) : Traverses a graph depth first.

Breadth First Search (BFS) : Traverses a graph level by level.

Dijkstra's Algorithm : Finds the shortest path from a source vertex.

Prim's Algorithm : Finds a Minimum Spanning Tree (MST).

Kruskal's Algorithm : Finds a Minimum Spanning Tree (MST).

09*Dijkstra's Algorithm (Shortest Path)

Definition:

Dijkstra's Algorithm finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights.

```
import heapq
def dijkstra(graph, start):
    distance = {vertex: float('inf') for vertex in graph}
    distance[start] = 0
    priority_queue = [(0, start)]
    while priority_queue:
        current_distance, current_vertex = heapq.heappop(priority_queue)
        if current_distance > distance[current_vertex]:
            continue
        for neighbor, weight in graph[current_vertex]:
            new_distance = current_distance + weight
            if new_distance < distance[neighbor]:
                distance[neighbor] = new_distance
                heapq.heappush(priority_queue, (new_distance, neighbor))
    return distance
```

Example

```
graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('D', 5)],
    'C': [('B', 1), ('D', 8)],
    'D': []
}
result = dijkstra(graph, 'A')
for vertex in result:
    print(vertex, ":", result[vertex])
```

Output

```
A : 0
B : 3
C : 2
D : 8
```

Chapter 19 : HashTable & Dictionary

01*What is a Hash Table?

A Hash Table is a data structure that stores data as key-value pairs. It uses a hash function to map each key to an index, allowing fast insertion, searching, and deletion.

Concept Example:

```
student = {
    "ID": 101,
    "Name": "John",
    "Grade": 95
}
print(student["Name"])
```

Output:

```
John
```

02*Why Dictionaries are Fast?

Python dictionaries are fast because they are implemented using Hash Tables.

Advantages:

- Average lookup time is $O(1)$.
- Fast insertion and deletion.
- Keys are mapped directly to their storage location using a hash function.
- Efficient for storing and retrieving key-value pairs.

03*Hash Function:

A Hash Function converts a key into a numeric value called a hash, which is used to determine where the key-value pair is stored in a hash table.

```
print(hash("apple"))  
print(hash("banana"))
```

Sample Output:

```
-320415278624...  
815932147586...
```

04*Collision in Python Dictionaries:

A collision occurs when two different keys produce the same hash index. Python automatically handles collisions internally, allowing both key-value pairs to be stored correctly.

```
student = {  
    "ID": 101,  
    "Name": "Alice",  
    "Grade": 95  
}  
print(student["Name"])
```

Output:

```
Alice
```

05*Creating a Dictionary:

A dictionary is created using curly braces {} with key-value pairs.

```
student = {  
    "ID": 101,  
    "Name": "John",  
    "Grade": 95  
}  
print(student)
```

Output:

```
{'ID': 101, 'Name': 'John', 'Grade': 95}
```

06*Inserting Data into a Dictionary:

New data is inserted by assigning a value to a new key.

```
student = {}  
student["ID"] = 101  
student["Name"] = "John"  
student["Grade"] = 95  
print(student)
```

Output:

```
{'ID': 101, 'Name': 'John', 'Grade': 95}
```

07*Retrieving Data from a Dictionary:

Dictionary values can be retrieved using their keys.

```
student = {  
    "ID": 101,  
    "Name": "John",  
    "Grade": 95  
}  
print(student["Name"])  
print(student["Grade"])
```

Output:

```
John  
95
```

08*Checking Keys and Values:

You can check whether a key or a value exists in a dictionary.

```
student = {  
    "ID": 101,  
    "Name": "John",  
    "Grade": 95  
}  
print("Name" in student)  
print(95 in student.values())
```

Output:

```
True  
True
```

09*Removing Data:

Data can be removed from a dictionary using the pop() method.

```
student = {  
    "ID": 101,  
    "Name": "John",  
    "Grade": 95  
}  
student.pop("Grade")  
print(student)
```

Output:

```
{'ID': 101, 'Name': 'John'}
```

10*Iterating Over a Dictionary:

A dictionary can be traversed using a for loop.

```
student = {  
    "ID": 101,  
    "Name": "John",  
    "Grade": 95  
}  
for key, value in student.items():  
    print(key, ":", value)
```

Output:

```
ID : 101  
Name : John  
Grade : 95
```

11*Null in Python Dictionary:

The keyword None represents a null value. A dictionary key can store None as its value.

```
student = {  
    "ID": 101,  
    "Name": None  
}  
print(student)
```

Output:

```
{'ID': 101, 'Name': None}
```

12*When to Use a Dictionary:

Use a dictionary when data needs to be accessed quickly using a unique key.

Common Uses:

- Storing student records.
- Managing user accounts.
- Counting word frequencies.
- Caching data.
- Storing configuration settings.